

Virtual sequence

Multi-agent env gives you several sequencers, one per active agent

Virtual sequencer vs virtual sequence

- The virtual sequencer is a container
- In connect phase the env wires those refs from each agent's sequencer
- Calls start on sub-sequences
- Sequential mode runs UART first, then SPI
- Parallel mode uses fork-join so both start at once
- Missing or null refs fail at start time

Browser lab

Digital Design and Verification Platform

HomeTools

Home / Tools / Virtual sequence

INTERACTIVE TOOL

Virtual sequence

Coordinate stimulus across agents: a **virtual sequencer** holds refs; a **virtual sequence** starts sub-sequences **sequentially** or in **parallel**. Starter: UART then SPI — both refs wired, sequential start done.

Starter example: virtual sequencer holds uart_sqr and spi_sqr; virtual sequence started them **sequentially** (UART then SPI).

Load starter example

Challenges 0 / 22 cleared

Quiz: vsequencer: A virtual sequencer primarily...

☐ holds references to agent sequencers (does not create them)

☐ replaces the DUT netlist

☐ only dumps VCD files

☐ sets the timescale pragma

Show hint

Check

Next

Idle

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

Core ideas

vsequencer

Holds refs to agent sequencers — does not create them.

vsequence

Starts sub-sequences on those refs.

sequential

Ordered start() — wait for one, then next.

parallel

fork-join starts both agents together.

Lab

Virtual sequence lab starter

Real UVM literacy

```
❯ u1 - module17-uvm-vseq/examples/vseq-sketch

# Track A ~ real Unix
# real shell session (Track A ~ virtual sequence)

$ pwd
/mnt/d/proj/designs/Learning/courses/Learn_uvm2017/module17-uvm-vseq

$ ls examples/vseq-sketch/
vseq.txt

$ cat examples/vseq-sketch/vseq.txt
Virtual sequence UVM sketch (literacy)

Roles:
virtual sequencer = holds handles to agent sequencers (does not create them)
virtual sequence = coordinates sub-sequences across those handles

Hierarchy (typical):
uvm_test
├── uvm_env
│   ├── uart_agent.sequencer
│   ├── spi_agent.sequencer
│   └── (optional) virtual_sequencer * often owned by env or test
│       ├── uart_sqr (ref)
│       └── spi_sqr (ref)

connect_phase (env or test):
vsqr.uart_sqr = env.uart_agent.sequencer;
vsqr.spi_sqr = env.spi_agent.sequencer;

virtual sequence body ~ sequential:
uart_seq seq_u;
spi_seq seq_s;
seq_u = uart_seq::type_id::create("seq_u");
seq_s = spi_seq::type_id::create("seq_s");
seq_u.start(vsqr.uart_sqr); // UART first
seq_s.start(vsqr.spi_sqr); // then SPI

virtual sequence body ~ parallel:
fork
  seq_u.start(vsqr.uart_sqr);
  seq_s.start(vsqr.spi_sqr);
join

test start (typical):
vseq = my_vseq::type_id::create("vseq");
vseq.start(vsqr); // vseq grabs refs from vsqr inside body/pre_body

Why virtual sequences:
one test coordinates multi-interface scenarios
reuse agent sequences instead of duplicating protocol code
choose sequential vs parallel timing explicitly

Common mistakes:
virtual sequencer expected to build agents
null ref (forgot connect) * start fails
driving pins inside vseq instead of delegating to agent sequences
parallel fork-join without checking scoreboard / ordering assumptions

Legacy reference:
learn_uvm2017_sv_verilator/module5/examples/virtual_sequences/virtual_sequences.sv

$ grep -n "VirtualSequencer\|VirtualSequence\|fork\|join"
.../learn_uvm2017_sv_verilator/module5/examples/virtual_sequences/virtual_sequences.sv | head -14
6: * 3. Master parallel sequence execution (fork-join)
25: * ~ Parallel: fork-join (sequences run simultaneously)
93: class VirtualSequence extends uvm_sequence;
97:   uvm_object_utils(VirtualSequence);
99:   function new(string name = "VirtualSequence");
114:   // ~ Uses fork-join construct
119:   // ~ fork: Start all sequences in parallel
120:   // ~ join: Wait for all sequences to complete
131:   // fork
132:   // join
142:   // ~ join waits for all to complete
144:   fork
172:   // ~ Both sequences complete before join continues
178:   join // Wait for both sequences to complete
```

Real shell — virtual sequence sketch

Pitfalls to watch

- Do not expect the virtual sequencer to build agents
- Do not start a sub-sequence on a null ref, check wiring before start
- Do not assume sequential and parallel behave the same
- Keep pin wiggling in drivers
- And remember

Your turn

- Complete the checklist for at least one track, preferably both
- In the browser
- Sketch virtual sequencer refs and one virtual sequence body with two start calls
- When you are ready, take the short quiz, then continue to callbacks in the next module